

Dual Quaternions for VR Poses and Blending

K.Chandran

Roll Number: NA23B040

AM5011 — Virtual Reality Engineering, Assignment 2

Unity 2022.3.52f1

14th September 2026

Abstract

This report derives the unit dual quaternion representation of a rigid 6-DOF pose, implements it as a reusable Unity C# component, and uses it to compare dual-quaternion pose blending against decoupled position/rotation interpolation. The dual-quaternion blend is shown, both analytically and through a numerical experiment, to follow a coupled screw-motion path rather than the straight chord produced by interpolating position and rotation independently.

1 Introduction

Rigid-body poses in VR — controller transforms, tracked limbs, networked avatar updates — are usually represented as a rotation and a translation kept in separate variables. Interpolating the two independently is simple but physically inconsistent: it does not correspond to any single rigid motion of the object between the two poses. A *unit dual quaternion* instead encodes rotation and translation as one coupled algebraic object, so that interpolating it produces a genuine rigid motion at every step. This report covers three things: the derivation of the dual quaternion pose representation and its inverse (Section 2), a Unity C# implementation of it (Section 3), and a small experiment comparing it against decoupled interpolation (Section 4).

2 Mathematical Formulation

2.1 Constructing the unit dual quaternion

Let q_r be a unit rotation quaternion ($\|q_r\| = 1$) and let $t = (t_x, t_y, t_z)$ be a translation vector, embedded as the pure quaternion $T = (t_x, t_y, t_z, 0)$. A dual quaternion is written $\hat{q} = q_r + \epsilon q_d$ with $\epsilon^2 = 0$.

A pure rotation has zero dual part, $\hat{R} = q_r + \epsilon \cdot 0$, and a pure translation is written $\hat{T} = 1 + \frac{1}{2}\epsilon T$. Applying the rotation first and the translation second gives the composite pose $\hat{q} = \hat{T}\hat{R}$:

$$\hat{q} = (1 + \frac{1}{2}\epsilon T) (q_r + 0 \cdot \epsilon) = q_r + \epsilon (\frac{1}{2}Tq_r).$$

This gives the pose representation used throughout the rest of the report:

$$\hat{q} = q_r + \epsilon q_d, \quad q_d = \frac{1}{2}Tq_r. \quad (1)$$

Since $\|q_r\| = 1$ and $q_r \cdot q_d = \text{Re}(q_d q_r^*) = \text{Re}(\frac{1}{2}T) = 0$, $\hat{q}$ satisfies both constraints of a unit dual quaternion and validly encodes the 6-DOF pose (R, t) .

2.2 Recovering rotation and translation

The real part directly gives the rotation, renormalized to remove floating-point drift:

$$R = \frac{q_r}{\|q_r\|}.$$

To recover the translation, start from $q_d = \frac{1}{2}Tq_r$ in Equation (1) and right-multiply by q_r^{-1} . Since q_r is unit, $q_r^{-1} = q_r^*$, so

$$2q_dq_r^* = T(q_rq_r^*) = T \quad \implies \quad T = 2q_dq_r^*.$$

The vector part of the resulting pure quaternion T is the recovered translation $t = (T_x, T_y, T_z)$; its scalar component is zero up to floating-point error.

2.3 Composition

Two poses compose as

$$\hat{q}_a\hat{q}_b = (q_{r,a}q_{r,b}) + \epsilon(q_{r,a}q_{d,b} + q_{d,a}q_{r,b}),$$

which applies b first, then a : $R = R_aR_b$ and $t = R_at_b + t_a$, matching Unity’s own quaternion/transform composition order.

2.4 Why dual quaternions for VR blending

Natural VR motions — a controller swinging through an arc, a shoulder, elbow and wrist chain articulating together — combine rotation and translation along a single curved path rather than two independent straight-line channels. Blending position and rotation separately (`Vector3.Lerp` together with `Quaternion.Slerp`) forces the object along a flat chord between the two endpoints while the orientation turns independently of it, which is the same decoupling responsible for “candy-wrapper” mesh collapse in skinning and for visibly unnatural sliding motion on a single rigid object. A dual quaternion blends translation and rotation as one coupled quantity, so the interpolated pose instead follows the true spatial screw motion between the two poses (Chasles’ theorem).

3 Implementation

The representation above was implemented in Unity C# as a `DualQuaternion` struct with two `Quaternion` fields, `r` (real part) and `d` (dual part), and the following methods:

- `FromPose(rotation, translation)` builds $\hat{q} = q_r + \epsilon(\frac{1}{2}Tq_r)$, Equation (1).
- `ToPose(out rotation, out translation)` extracts $R = q_r/\|q_r\|$ and $t = 2q_dq_r^*$.
- `Multiply(a, b)` performs the rigid composition derived above.
- `Slerp(a, b, u)` blends two poses: shortest-path sign alignment on the real part, `Quaternion.Slerp` on q_r , a componentwise `Mathf.Lerp` on q_d , then re-normalization. This is dual quaternion linear blending (DLB), an approximation to true constant-speed screw interpolation (ScLERP) that is standard practice for this kind of assignment.
- `Normalized()` and `Conjugate()` enforce $\|q_r\| = 1$ and $q_r \cdot q_d = 0$, and provide the dual quaternion conjugate used internally.

Two details were easy to get wrong and are worth recording. First, Unity’s `Quaternion` type has no operator for addition or for scalar multiplication between two quaternions, so every dual-part operation had to be written out componentwise rather than through `Quaternion` arithmetic. Second, the dual part of the identity pose is the *zero* quaternion $(0, 0, 0, 0)$, not `Quaternion.identity` (which is $(0, 0, 0, 1)$); using the latter would silently encode a phantom translation. For the same reason, `Slerp` blends the dual part with `Mathf.Lerp` component-by-component rather than `Quaternion.Lerp`, which renormalizes its result to a unit quaternion and would corrupt the generally non-unit dual part.

3.1 Demo scene

An editor script adds the menu command **AM5011** $\rightarrow$ **Build Full Demo Scene**, which constructs the scene programmatically: a fixed `PoseA` and `PoseB` anchor, an `InterpolatedPose` object driven by `DualQuaternion.Slerp`, a `DecoupledPose` object driven by plain `Vector3.Lerp` and `Quaternion.Slerp` for comparison, a UI slider bound to the interpolation parameter u , and a `TrailRenderer` on each moving object to visualize the swept path.

A separate Play-mode test component checks that `FromPose/ ToPose` round-trips correctly, that `Slerp` exactly recovers both endpoints at $u = 0, 1$, that the blended pose remains a valid unit dual quaternion at $u = 0.5$, and that `Multiply` composes two poses the same way Unity’s own transform chaining does.

4 Experimental Evaluation

4.1 Setup

To isolate the effect of coupling, `PoseA` was placed at $(-2, 0, 0)$ m with identity rotation and `PoseB` at $(2, 0, 0)$ m rotated 120° about $+Z$. Both interpolation methods were evaluated on this identical pair of endpoints: (1) *separate* interpolation (`Vector3.Lerp` on position, `Quaternion.Slerp` on rotation), and (2) *dual-quaternion* interpolation (`DualQuaternion.Slerp(dqA, dqB, u)`). The underlying math was re-implemented independently in Python to sample both paths at 101 values of $u \in [0, 1]$ and generate Figure 1 and Table 1.

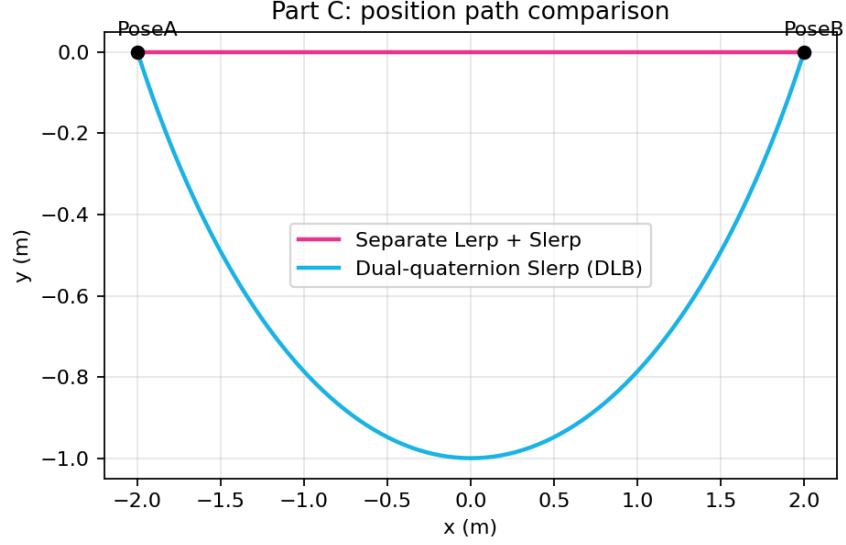


Figure 1: Position path of the two interpolation methods between the same pair of endpoints.

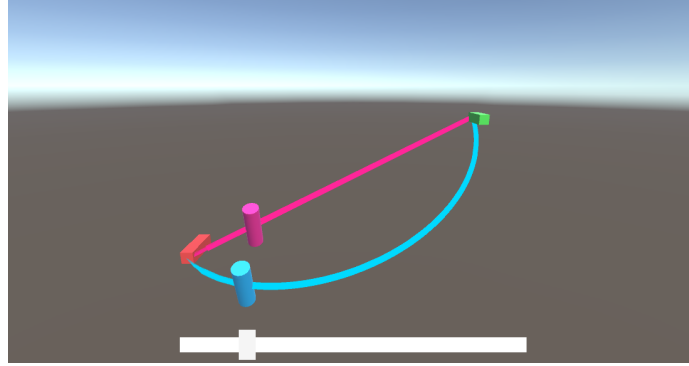


Figure 2: Unity demo scene: **PoseA** (red) and **PoseB** (green) anchors with the swept trails of **InterpolatedPose** (cyan, dual quaternion) and **DecoupledPose** (magenta, separate Lerp+Slerp), the same curved-versus-straight distinction as Figure 1 reproduced live in the Editor.

u	DLB x (m)	DLB y (m)	DLB angle	Separate angle
0.00	-2.000	0.000	0.00°	0.00°
0.25	-1.095	-0.742	30.00°	30.00°
0.50	0.000	-1.000	60.00°	60.00°
0.75	1.095	-0.742	90.00°	90.00°
1.00	2.000	0.000	120.00°	120.00°

Table 1: Position and orientation of both methods at five sample values of u .

4.2 Observations

- **Curved vs. straight path.** The separate method’s position follows the straight chord $y = 0$ between the anchors, while the dual-quaternion blend follows a curved arc dipping to $y = -1.0$ m at $u = 0.5$. Coupling rotation into the translation term ($q_d = \frac{1}{2}Tq_r$) is what bends the path.
- **Orientation timing is unaffected here.** Both methods compute rotation as `Quaternion.Slerp` of the same endpoint quaternions, so the angle-versus- u curve matches exactly in this experiment; the two methods diverge only in where the translation goes, not in rotation timing. For a genuine screw motion, this position/rotation coupling is exactly what decoupled interpolation cannot express.
- **Naturalness for VR.** The curved path is consistent with a joint (shoulder, elbow, wrist) sweeping a controller through an arc; the straight-chord path looks mechanical and unnatural for a rotating hand or tool, since it is only correct when translation and rotation happen to be physically independent.
- **Rigidity is preserved either way.** Because `Normalized()` is applied after every blend, the dual-quaternion interpolation stays on the unit dual-quaternion manifold throughout $u \in [0, 1]$, with no collapse or scale drift at the midpoint.
- **Caveat.** This simple DLB (linear blend followed by re-normalization) approximates true constant-speed ScLERP; for larger rotation angles or more widely separated anchors the coupling becomes even more pronounced, which is the same effect exploited to reduce skinning artifacts in avatar deformation (Kavan et al., 2008).

5 Conclusion

Representing a 6-DOF pose as a unit dual quaternion couples rotation and translation into a single interpolable object. The derivation in Section 2 shows this representation is exact and invertible; the implementation in Section 3 shows it fits cleanly into Unity’s existing `Quaternion/Vector3` API once the componentwise dual-part arithmetic is handled correctly; and the experiment in Section 4 shows the practical consequence of that coupling: a curved, physically coherent motion path in place of the straight chord produced by interpolating position and rotation separately. This is the property that makes dual quaternions useful for VR controller smoothing, avatar skinning, and networked pose updates.

Appendix: Code and Data Listing

- `Assets/Scripts/DualQuaternion.cs`, `DualQuaternionPoseDemo.cs`, `Editor/SceneSetupHelper.cs`, `DualQuaternionMathTests.cs` — Unity implementation.
- `tools/dualquat_sim.py` — independent numeric check and generator for Figure 1 and Table 1.
- This report (`report/main.tex`).